CORRIGÉ · PROBLÈME · ANNALES SPÉ

Graphes en Python et requêtes SQL

Énoncé

Les graphes sont orientés, de sommets numérotés à partir de 0, et représentés par un dictionnaire d'adjacence. Exemple : $d = \{0 : [1, 3],\ 1 : [2],\ 2 : [4, 5],\ 3 : [4],\ 4 : [1],\ 5 : [\,],\ 6 : [2]\}$.

1. Écrire `degreMax(d)` qui renvoie le degré sortant maximal.
2. Le graphe inverse a les mêmes sommets et les arêtes en sens inverse. Représenter le graphe inverse de l'exemple, puis écrire `grapheInv(d)`.
3. Une coloration L est valide si deux sommets reliés par une arête n'ont pas la même couleur $L[s]$. Écrire `colorationValide(d, L)`.
4. Donner sa complexité dans le pire des cas en fonction du nombre N de sommets et M d'arêtes.

Tables `FILMS(codefilm, nomfilm)` et `LOCATIONS(codecli, codefilm, datedebut, duree)`.

1. Écrire une requête donnant la plus grande durée de location.
2. Écrire une requête donnant le code, le nom et la durée moyenne de location des films loués en moyenne moins de 2 jours, par durée moyenne décroissante.

Décoder l'énoncé

Ce que l'énoncé te donne.

- `d[s]` est la liste des successeurs de `s`.

Ce qu'on te demande. Parcourir un dictionnaire d'adjacence ; écrire deux requêtes.

Corrigé

Question 1

Étape 1. Q1. Le degré sortant maximal

Le degré sortant d'un sommet `s` est le nombre de ses successeurs, `len(d[s])`. On parcourt tous les sommets en gardant le plus grand :

```
def degreMax(d):
    m = 0
    for s in d:              # s parcourt les clés (les sommets)
```

```
        if len(d[s]) > m:
            m = len(d[s])
    return m
```

Sur l'exemple : les degrés sont $2, 1, 2, 1, 1, 0, 1$, la fonction renvoie 2.

Question 2

Étape 1. Q2. Le graphe inverse

Arêtes de l'exemple : $0 \to 1$, $0 \to 3$, $1 \to 2$, $2 \to 4$, $2 \to 5$, $3 \to 4$, $4 \to 1$, $6 \to 2$. On les retourne : $1 \to 0$, $3 \to 0$, $2 \to 1$, $4 \to 2$, $5 \to 2$, $4 \to 3$, $1 \to 4$, $2 \to 6$. Dictionnaire du graphe inverse :

$$\{0 : [\,],\ 1 : [0, 4],\ 2 : [1, 6],\ 3 : [0],\ 4 : [2, 3],\ 5 : [2],\ 6 : [\,]\}.$$

```
def grapheInv(d):
    inv = {s: [] for s in d}   # mêmes sommets, aucune arête au départ
    for s in d:
        for t in d[s]:          # arête s -> t
            inv[t].append(s)    # devient t -> s
    return inv
```

Question 3

Étape 1. Q3. Vérifier une coloration

Une coloration est invalide dès qu'une arête relie deux sommets de même couleur. On teste toutes les arêtes :

```
def colorationValide(d, L):
    for s in d:
        for t in d[s]:
            if L[s] == L[t]:
                return False   # une arête monochrome suffit
    return True
```

Question 4

Étape 1. Q4. Complexité

La boucle extérieure fait N tours. La boucle intérieure, au total sur tous les sommets, parcourt chaque arête une fois : M tours au total. Chaque tour coûte un temps constant (accès à une liste, comparaison). Dans le pire des cas (coloration valide, on va jusqu'au bout) :

complexité $O(N + M)$.

Question 5

Étape 1. Q5. La plus grande durée

```
SELECT MAX(duree) FROM LOCATIONS;
```

Question 6

Étape 1. Q6. Durées moyennes, avec jointure

Il faut le nom du film (table `FILMS`) et les durées (table `LOCATIONS`) : on fait une **jointure** sur `codefilm`. On regroupe par film, on calcule la moyenne, on filtre les groupes avec `HAVING` (une condition sur un agrégat ne peut pas être dans `WHERE`), puis on trie :

```
SELECT F.codefilm, F.nomfilm, AVG(L.duree) AS moy
FROM FILMS F JOIN LOCATIONS L ON F.codefilm = L.codefilm
GROUP BY F.codefilm, F.nomfilm
HAVING AVG(L.duree) < 2
ORDER BY moy DESC;
```

Le piège

En Q6, mettre la condition sur `AVG` dans `WHERE` : `WHERE` filtre les lignes avant le regroupement.

La méthode

Parcourir un graphe par dictionnaire : une double boucle « sommet, puis successeurs » visite chaque arête une fois.